Skill

Topic 1 : To Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Directory, Test Navigation

```
vennela_07@VennelaPC:~$ mkdir directory_navigation_project
cd directory_navigation_project
vennela_07@VennelaPC:~/directory_navigation_project$ nano directory_navigation.c
vennela_07@VennelaPC:~/directory_navigation_project$ gcc -Wall -Wextra directory_navigation.c -o directory_navigation
vennela_07@VennelaPC:~/directory_navigation_project$ ./directory_navigation
Directory Navigation Program
Commands: cd PATH, pwd, cd -, exit

/home/vennela_07/directory_navigation_project> pwd
Current directory: /home/vennela_07/directory_navigation_project

/home/vennela_07/directory_navigation_project> cd ..
pwdDirectory changed successfully.
Current directory: /home/vennela_07

/home/vennela_07>
Current directory: /home/vennela_07

/home/vennela_07> cd -
Directory changed successfully.
Current directory: /home/vennela_07/directory_navigation_project

/home/vennela_07/directory_navigation_project> cd ~
Directory changed successfully.
Current directory: /home/vennela_07

/home/vennela_07> cd /invalid/path
Path validation failed: No such file or directory

/home/vennela_07> cd /etc/hosts
Error: Path is not a directory.

/home/vennela_07> exit
Directory navigation terminated.
```

```
  GNU nano 7.2                                                directory_navigation.c

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/stat.h>
#include <limits.h>
#include <errno.h>

#ifndef PATH_MAX
#define PATH_MAX 4096
#endif

#define INPUT_SIZE 512

int main(void) {
    char input[INPUT_SIZE];
    char current_dir[PATH_MAX];
    char previous_dir[PATH_MAX] = "";
    char target[PATH_MAX];

    if (getcwd(current_dir, sizeof(current_dir)) == NULL) {
        perror("getcwd");
        return 1;
    }

    printf("Directory Navigation Program\n");
    printf("Commands: cd PATH, pwd, cd -, exit\n");

    while (1) {
        printf("\n%s> ", current_dir);
        fflush(stdout);
```

```
        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        if (strchr(input, '\n') == NULL && !feof(stdin)) {
            int ch;
            while ((ch = getchar()) != '\n' && ch != EOF) {}
            printf("Error: Input too long.\n");
            continue;
        }

        input[strcspn(input, "\n")] = '\0';

        char *command = strtok(input, " \t");
        char *argument = strtok(NULL, " \t");
        char *extra = strtok(NULL, " \t");

        if (command == NULL)
            continue;

        if (strcmp(command, "exit") == 0)
            break;

        if (strcmp(command, "pwd") == 0) {
            if (argument != NULL) {
                printf("Usage: pwd\n");
                continue;
            }

            printf("Current directory: %s\n", current_dir);
            continue;
        }
```

```
        if (strcmp(command, "cd") != 0) {
            printf("Error: Unknown command.\n");
            continue;
        }

        if (extra != NULL) {
            printf("Usage: cd PATH\n");
            continue;
        }

        if (argument == NULL) {
            argument = getenv("HOME");

            if (argument == NULL || argument[0] == '\0') {
                printf("Error: HOME is not set.\n");
                continue;
            }
        }

        if (strcmp(argument, "-") == 0) {
            if (previous_dir[0] == '\0') {
                printf("Error: No previous directory.\n");
                continue;
            }

            strcpy(target, previous_dir);
        } else if (argument[0] == '~' &&
                   (argument[1] == '/' || argument[1] == '\0')) {
            const char *home = getenv("HOME");
```

```
            if (home == NULL) {
                printf("Error: HOME is not set.\n");
                continue;
            }

            int result = snprintf(target, sizeof(target), "%s%s",
                                  home, argument + 1);

            if (result < 0 || (size_t)result >= sizeof(target)) {
                printf("Error: Path is too long.\n");
                continue;
            }
        } else {
            int result = snprintf(target, sizeof(target), "%s",
                                  argument);

            if (result < 0 || (size_t)result >= sizeof(target)) {
                printf("Error: Path is too long.\n");
                continue;
            }
        }

        struct stat info;

        if (stat(target, &info) != 0) {
            perror("Path validation failed");
            continue;
        }

        if (!S_ISDIR(info.st_mode)) {
            printf("Error: Path is not a directory.\n");
            continue;
        }
          char new_dir[PATH_MAX];

          if (getcwd(new_dir, sizeof(new_dir)) == NULL) {
              perror("getcwd");
              continue;
          }

          strcpy(previous_dir, current_dir);
          strcpy(current_dir, new_dir);

          printf("Directory changed successfully.\n");
          printf("Current directory: %s\n", current_dir);
    }

    printf("Directory navigation terminated.\n");
    return 0;
}
```

Topic 2 : To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic

```
vennela_07@VennelaPC:~$ mkdir builtin_dispatch_project
cd builtin_dispatch_project
mkdir: cannot create directory ‘builtin_dispatch_project’: File exists
vennela_07@VennelaPC:~/builtin_dispatch_project$ nano builtin_dispatch.c
vennela_07@VennelaPC:~/builtin_dispatch_project$ gcc -Wall -Wextra builtin_dispatch.c -o builtin_dispatch
vennela_07@VennelaPC:~/builtin_dispatch_project$ ./builtin_dispatch
Built-in Command Shell
Type 'help' for commands.
shell> help
Available built-in commands:
help  - Display available commands
cd    - Change current directory
pwd   - Display current directory
set   - Set a shell variable
get   - Display a shell variable
exit  - Exit the shell
shell> pwd
/home/vennela_07/builtin_dispatch_project
shell> cd ..
pwdshell>
/home/vennela_07
shell>
shell> set COURSE OS
geVariable COURSE set successfully.
shell> t COURSE
COURSE=OS
shell>
shell> hello
Invalid command: hello
Type 'help' to see available commands.
shell> exit
Shell terminated.
```

```
  GNU nano 7.2                                                builtin_dispatch.c

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>

#define INPUT_SIZE 512
#define MAX_ARGS 20

typedef int (*CommandFunction)(char **args);

typedef struct {
    const char *name;
    CommandFunction function;
} BuiltinCommand;

/* Built-in command functions */

int command_help(char **args) {
    (void)args;

    printf("Available built-in commands:\n");
    printf("help  - Display available commands\n");
    printf("cd    - Change current directory\n");
    printf("pwd   - Display current directory\n");
    printf("set   - Set a shell variable\n");
    printf("get   - Display a shell variable\n");
    printf("exit  - Exit the shell\n");

    return 0;
}
```

Topic 3 : To Retrieve Current Directory, Display Path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior

```
vennela_07@VennelaPC:~$ cd ~/directory_state_project
vennela_07@VennelaPC:~/directory_state_project$ nano directory_state.c
vennela_07@VennelaPC:~/directory_state_project$ gcc -Wall -Wextra directory_state.c -o directory_state
vennela_07@VennelaPC:~/directory_state_project$ ./directory_state
Directory State Management Program
Current working directory: /home/vennela_07/directory_state_project

Enter command (pwd, save, exit): pwd save
Invalid command. Use pwd, save, or exit.

Enter command (pwd, save, exit): exit
Final directory state saved.
Cleaning up resources...
Program terminated successfully.
Cleanup complete.
vennela_07@VennelaPC:~/directory_state_project$ cat directory_state.txt
/home/vennela_07/directory_state_project
vennela_07@VennelaPC:~/directory_state_project$ ls
directory_state  directory_state.c  directory_state.txt
```

```
  GNU nano 7.2                                                    directory_state.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>
#include <errno.h>

#ifndef PATH_MAX
#define PATH_MAX 4096
#endif

#define INPUT_SIZE 512

int save_state(const char *path) {
    FILE *file = fopen("directory_state.txt", "w");

    if (file == NULL) {
        perror("Unable to save state");
        return 0;
    }

    if (fprintf(file, "%s\n", path) < 0) {
        perror("Unable to write state");
        fclose(file);
        return 0;
    }

    if (fclose(file) != 0) {
        perror("Unable to close state file");
        return 0;
    }

    return 1;
}

int main(void) {
    char cwd[PATH_MAX];
    char input[INPUT_SIZE];

    printf("Directory State Management Program\n");

    if (getcwd(cwd, sizeof(cwd)) == NULL) {
        perror("getcwd failed");
        return 1;
    }
```

```c
    printf("Current working directory: %s\n", cwd);

    while (1) {
        printf("\nEnter command (pwd, save, exit): ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL) {
            printf("\nInput closed. Cleaning up...\n");
            break;
        }

        if (strchr(input, '\n') == NULL && !feof(stdin)) {
            int ch;

            while ((ch = getchar()) != '\n' && ch != EOF) {}

            printf("Error: Input too long.\n");
            continue;
        }

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "pwd") == 0) {
            if (getcwd(cwd, sizeof(cwd)) == NULL) {
                perror("getcwd failed");
                continue;
            }

            printf("Current directory: %s\n", cwd);
        } else if (strcmp(input, "save") == 0) {
            if (getcwd(cwd, sizeof(cwd)) == NULL) {
                perror("getcwd failed");
                continue;
            }

            if (save_state(cwd))
                printf("Directory state saved successfully.\n");
        } else if (strcmp(input, "exit") == 0) {
            if (getcwd(cwd, sizeof(cwd)) != NULL) {
                if (save_state(cwd))
                    printf("Final directory state saved.\n");
            } else {
                perror("getcwd failed during cleanup");
            }
            }

            printf("Cleaning up resources...\n");
            printf("Program terminated successfully.\n");
            break;
        } else if (input[0] == '\0') {
            continue;
        } else {
            printf("Invalid command. Use pwd, save, or exit.\n");
        }
    }

    printf("Cleanup complete.\n");
    return 0;
}
```

Topic 4 : To Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality

```
vennela_07@VennelaPC:~$ cd ~
mkdir export_project
cd export_project
vennela_07@VennelaPC:~/export_project$ nano export_command.c
vennela_07@VennelaPC:~/export_project$ gcc -Wall -Wextra export_command.c -o export_command
vennela_07@VennelaPC:~/export_project$ ./export_command

Enter export command (or exit): export COURSE=OSSP
Variable updated successfully.
COURSE=OSSP
SHELL=/bin/bash
WSL2_GUI_APPS_ENABLED=1
WSL_DISTRO_NAME=Ubuntu
NAME=VennelaPC
PWD=/home/vennela_07/export_project
LOGNAME=vennela_07
HOME=/home/vennela_07
LANG=C.UTF-8
WSL_INTEROP=/run/WSL/1359_interop
LS_COLORS=rs=0:di=01;34:ln=01;36:mh=00:pi=40;33:so=01;35:do=01;35:bd=40;33;01:cd=40;33;01:or=40;31;01:mi=00:su=37;41:sg=30;43:ca=00:t
w=30;42:ow=34;42:st=37;44:ex=01;32:*.tar=01;31:*.tgz=01;31:*.arc=01;31:*.arj=01;31:*.taz=01;31:*.lha=01;31:*.lz4=01;31:*.lzh=01;31:*.
lzma=01;31:*.tlz=01;31:*.txz=01;31:*.tzo=01;31:*.t7z=01;31:*.zip=01;31:*.z=01;31:*.dz=01;31:*.gz=01;31:*.lrz=01;31:*.lz=01;31:*.lzo=0
1;31:*.xz=01;31:*.zst=01;31:*.tzst=01;31:*.bz2=01;31:*.bz=01;31:*.tbz=01;31:*.tbz2=01;31:*.tz=01;31:*.deb=01;31:*.rpm=01;31:*.jar=01;
31:*.war=01;31:*.ear=01;31:*.sar=01;31:*.rar=01;31:*.alz=01;31:*.ace=01;31:*.zoo=01;31:*.cpio=01;31:*.7z=01;31:*.rz=01;31:*.cab=01;31
:*.wim=01;31:*.swm=01;31:*.dwm=01;31:*.esd=01;31:*.avif=01;35:*.jpg=01;35:*.jpeg=01;35:*.mjpg=01;35:*.mjpeg=01;35:*.gif=01;35:*.bmp=0
1;35:*.pbm=01;35:*.pgm=01;35:*.ppm=01;35:*.tga=01;35:*.xbm=01;35:*.xpm=01;35:*.tif=01;35:*.tiff=01;35:*.png=01;35:*.svg=01;35:*.svgz=
01;35:*.mng=01;35:*.pcx=01;35:*.mov=01;35:*.mpg=01;35:*.mpeg=01;35:*.m2v=01;35:*.mkv=01;35:*.webm=01;35:*.webp=01;35:*.ogm=01;35:*.mp
4=01;35:*.m4v=01;35:*.mp4v=01;35:*.vob=01;35:*.qt=01;35:*.nuv=01;35:*.wmv=01;35:*.asf=01;35:*.rm=01;35:*.rmvb=01;35:*.flc=01;35:*.avi
=01;35:*.fli=01;35:*.flv=01;35:*.gl=01;35:*.dl=01;35:*.xcf=01;35:*.xwd=01;35:*.yuv=01;35:*.cgm=01;35:*.emf=01;35:*.ogv=01;35:*.ogx=01
;35:*.aac=00;36:*.au=00;36:*.flac=00;36:*.m4a=00;36:*.mid=00;36:*.midi=00;36:*.mka=00;36:*.mp3=00;36:*.mpc=00;36:*.ogg=00;36:*.ra=00;
36:*.wav=00;36:*.oga=00;36:*.opus=00;36:*.spx=00;36:*.xspf=00;36:*~=00;90:*#=00;90:*.bak=00;90:*.crdownload=00;90:*.dpkg-dist=00;90:*
.dpkg-new=00;90:*.dpkg-old=00;90:*.dpkg-tmp=00;90:*.old=00;90:*.orig=00;90:*.part=00;90:*.rej=00;90:*.rpmnew=00;90:*.rpmorig=00;90:*.
rpmsave=00;90:*.swp=00;90:*.tmp=00;90:*.ucf-dist=00;90:*.ucf-new=00;90:*.ucf-old=00;90:
WAYLAND_DISPLAY=wayland-0
LESSCLOSE=/usr/bin/lesspipe %s %s
TERM=xterm-256color
LESSOPEN=| /usr/bin/lesspipe %s
USER=vennela_07
DISPLAY=:0
SHLVL=1
XDG_RUNTIME_DIR=/run/user/1000
DEBUGINFOD_URLS=https://debuginfod.ubuntu.com
WSLENV=
XDG_DATA_DIRS=/usr/local/share:/usr/share:/var/lib/snapd/desktop
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/usr/lib/wsl/lib:/mnt/c/Program Files/E
clipse Adoptium/jdk-21.0.12.101-hotspot/bin:/mnt/c/Program Files/Common Files/Oracle/Java/javapath:/mnt/c/Program Files (x86)/Common
Files/Oracle/Java/java8path:/mnt/c/Program Files (x86)/Common Files/Oracle/Java/javapath:/mnt/c/windows/system32:/mnt/c/windows:/mnt/
c/windows/System32/Wbem:/mnt/c/windows/System32/WindowsPowerShell/v1.0/:/mnt/c/windows/System32/OpenSSH/:/mnt/c/Program Files/nodejs/
:/mnt/c/Users/samba/AppData/Local/Programs/Python/Python314/Scripts/:/mnt/c/Users/samba/AppData/Local/Programs/Python/Python314/:/mnt
/c/Users/samba/AppData/Local/Programs/Python/Launcher/:/mnt/c/Program Files/MySQL/MySQL Shell 8.0/bin/:/mnt/c/Users/samba/AppData/Loc
al/Microsoft/WindowsApps:/mnt/c/Users/samba/AppData/Local/Programs/Microsoft VS Code/bin:/mnt/c/Users/samba/AppData/Roaming/npm:/mnt/
c/Users/samba/AppData/Local/Programs/DockerDesktop/resources/bin:/snap/bin
DBUS_SESSION_BUS_ADDRESS=unix:path=/run/user/1000/bus
HOSTTYPE=x86_64
PULSE_SERVER=unix:/mnt/wslg/PulseServer
OLDPWD=/home/vennela_07
_=./export_command
COURSE=OSSP

Enter export command (or exit): export COURSE=LINUX
Variable updated successfully.
COURSE=LINUX
SHELL=/bin/bash
WSL2_GUI_APPS_ENABLED=1
WSL_DISTRO_NAME=Ubuntu
NAME=VennelaPC
PWD=/home/vennela_07/export_project
LOGNAME=vennela_07
HOME=/home/vennela_07
LANG=C.UTF-8
WSL_INTEROP=/run/WSL/1359_interop
LS_COLORS=rs=0:di=01;34:ln=01;36:mh=00:pi=40;33:so=01;35:do=01;35:bd=40;33;01:cd=40;33;01:or=40;31;01:mi=00:su=37;41:sg=30;43:ca=00:t
w=30;42:ow=34;42:st=37;44:ex=01;32:*.tar=01;31:*.tgz=01;31:*.arc=01;31:*.arj=01;31:*.taz=01;31:*.lha=01;31:*.lz4=01;31:*.lzh=01;31:*.
lzma=01;31:*.tlz=01;31:*.txz=01;31:*.tzo=01;31:*.t7z=01;31:*.zip=01;31:*.z=01;31:*.dz=01;31:*.gz=01;31:*.lrz=01;31:*.lz=01;31:*.lzo=0
1;31:*.xz=01;31:*.zst=01;31:*.tzst=01;31:*.bz2=01;31:*.bz=01;31:*.tbz=01;31:*.tbz2=01;31:*.tz=01;31:*.deb=01;31:*.rpm=01;31:*.jar=01;
31:*.war=01;31:*.ear=01;31:*.sar=01;31:*.rar=01;31:*.alz=01;31:*.ace=01;31:*.zoo=01;31:*.cpio=01;31:*.7z=01;31:*.rz=01;31:*.cab=01;31
:*.wim=01;31:*.swm=01;31:*.dwm=01;31:*.esd=01;31:*.avif=01;35:*.jpg=01;35:*.jpeg=01;35:*.mjpg=01;35:*.mjpeg=01;35:*.gif=01;35:*.bmp=0
1;35:*.pbm=01;35:*.pgm=01;35:*.ppm=01;35:*.tga=01;35:*.xbm=01;35:*.xpm=01;35:*.tif=01;35:*.tiff=01;35:*.png=01;35:*.svg=01;35:*.svgz=
01;35:*.mng=01;35:*.pcx=01;35:*.mov=01;35:*.mpg=01;35:*.mpeg=01;35:*.m2v=01;35:*.mkv=01;35:*.webm=01;35:*.webp=01;35:*.ogm=01;35:*.mp
4=01;35:*.m4v=01;35:*.mp4v=01;35:*.vob=01;35:*.qt=01;35:*.nuv=01;35:*.wmv=01;35:*.asf=01;35:*.rm=01;35:*.rmvb=01;35:*.flc=01;35:*.avi
=01;35:*.fli=01;35:*.flv=01;35:*.gl=01;35:*.dl=01;35:*.xcf=01;35:*.xwd=01;35:*.yuv=01;35:*.cgm=01;35:*.emf=01;35:*.ogv=01;35:*.ogx=01
;35:*.aac=00;36:*.au=00;36:*.flac=00;36:*.m4a=00;36:*.mid=00;36:*.midi=00;36:*.mka=00;36:*.mp3=00;36:*.mpc=00;36:*.ogg=00;36:*.ra=00;
36:*.wav=00;36:*.oga=00;36:*.opus=00;36:*.spx=00;36:*.xspf=00;36:*~=00;90:*#=00;90:*.bak=00;90:*.crdownload=00;90:*.dpkg-dist=00;90:*
.dpkg-new=00;90:*.dpkg-old=00;90:*.dpkg-tmp=00;90:*.old=00;90:*.orig=00;90:*.part=00;90:*.rej=00;90:*.rpmnew=00;90:*.rpmorig=00;90:*.
rpmsave=00;90:*.swp=00;90:*.tmp=00;90:*.ucf-dist=00;90:*.ucf-new=00;90:*.ucf-old=00;90:
WAYLAND_DISPLAY=wayland-0
LESSCLOSE=/usr/bin/lesspipe %s %s
TERM=xterm-256color
LESSOPEN=| /usr/bin/lesspipe %s
USER=vennela_07
DISPLAY=:0
SHLVL=1
```

```
SHLVL=1
XDG_RUNTIME_DIR=/run/user/1000
DEBUGINFOD_URLS=https://debuginfod.ubuntu.com
WSLENV=
XDG_DATA_DIRS=/usr/local/share:/usr/share:/var/lib/snapd/desktop
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/usr/lib/wsl/lib:/mnt/c/Program Files/E
clipse Adoptium/jdk-21.0.12.101-hotspot/bin:/mnt/c/Program Files/Common Files/Oracle/Java/javapath:/mnt/c/Program Files (x86)/Common
Files/Oracle/Java/java8path:/mnt/c/Program Files (x86)/Common Files/Oracle/Java/javapath:/mnt/c/windows/system32:/mnt/c/windows:/mnt/
c/windows/System32/Wbem:/mnt/c/windows/System32/WindowsPowerShell/v1.0/:/mnt/c/windows/System32/OpenSSH/:/mnt/c/Program Files/nodejs/
:/mnt/c/Users/samba/AppData/Local/Programs/Python/Python314/Scripts/:/mnt/c/Users/samba/AppData/Local/Programs/Python/Python314/:/mnt
/c/Users/samba/AppData/Local/Programs/Python/Launcher/:/mnt/c/Program Files/MySQL/MySQL Shell 8.0/bin/:/mnt/c/Users/samba/AppData/Loc
al/Microsoft/WindowsApps:/mnt/c/Users/samba/AppData/Local/Programs/Microsoft VS Code/bin:/mnt/c/Users/samba/AppData/Roaming/npm:/mnt/
c/Users/samba/AppData/Local/Programs/DockerDesktop/resources/bin:/snap/bin
DBUS_SESSION_BUS_ADDRESS=unix:path=/run/user/1000/bus
HOSTTYPE=x86_64
PULSE_SERVER=unix:/mnt/wslg/PulseServer
OLDPWD=/home/vennela_07
_=./export_command
COURSE=LINUX

Enter export command (or exit): export 123ABC=hello
Invalid variable name.

Enter export command (or exit): export COURSE
Invalid syntax. Use: export NAME=value
Invalid syntax. Use: export NAME=value

Enter export command (or exit): exit
Program terminated.
```

```
  GNU nano 7.2                                                export_command.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <sys/wait.h>
#include <unistd.h>

int valid_name(const char *name)
{
    int i;

    if (!isalpha((unsigned char)name[0]) && name[0] != '_')
        return 0;

    for (i = 1; name[i] != '\0'; i++)
    {
        if (!isalnum((unsigned char)name[i]) && name[i] != '_')
            return 0;
    }

    return 1;
}

int main(void)
{
    char input[256];
    char *name, *value, *equal;
    char *args[] = {"printenv", NULL};
    pid_t pid;

    while (1)
    {
        printf("\nEnter export command (or exit): ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';
```

```c
        if (strcmp(input, "exit") == 0)
            break;

        if (strncmp(input, "export ", 7) != 0)
        {
            printf("Invalid syntax. Use: export NAME=value\n");
            continue;
        }

        name = input + 7;
        equal = strchr(name, '=');

        if (equal == NULL)
        {
            printf("Invalid syntax. Use: export NAME=value\n");
            continue;
        }

        *equal = '\0';
        value = equal + 1;

        if (!valid_name(name))
        {
            printf("Invalid variable name.\n");
            continue;
        }

        if (setenv(name, value, 1) != 0)
        {
            perror("setenv");
            continue;
        }

        printf("Variable updated successfully.\n");
        printf("%s=%s\n", name, getenv(name));

        pid = fork();

        if (pid < 0)
        {
            perror("fork");
        }
        else if (pid == 0)
        {
            execvp(args[0], args);
            perror("execvp");
            _exit(1);
        }
        else
        {
          {
              waitpid(pid, NULL, 0);
          }
    }

    printf("Program terminated.\n");
    return 0;
}
```